

FitLens Project Spec

Project Name: FitLens

Repository Name Suggestion: fitlens-desktop

Project Type: Local Python desktop application with an initial command-line version

Project Framing: Historical fitness analysis and monthly coaching check-in tool

1. General Description of the Project

FitLens is a local fitness analysis application that helps a person understand their training history by combining data that is usually split across multiple apps. A person may use Apple Health for heart rate, HRV, resting heart rate, sleep, VO2 max, and body stats, while also using Hevy to track strength workouts, sets, reps, weight, and training volume. Each app shows part of the picture, but neither gives a clear monthly overview of how training, recovery, and progress fit together. FitLens brings those exports into one local application and turns them into historical trends, monthly summaries, and practical recommendations.

The project originally had a stronger focus on current fitness scores, including a readiness score. After thinking through the data flow, that does not make as much sense for this version because the app relies on bulk file uploads. If a user only imports new data once a month, a current readiness score is not very useful. Instead, FitLens will work more like a monthly coach check-in. The app will review the user's recent training block, compare it to past months, identify patterns, and suggest what the user may want to adjust going forward.

The first version of FitLens will use file imports rather than live API connections. The user will export their Apple Health data as an XML file and export their Hevy workout history as a CSV file. FitLens will parse both files, clean the data, match workouts from Hevy with heart-rate data from Apple Health during the same time windows when possible, and generate a monthly training report. The goal is not to diagnose health conditions or replace a doctor or coach. The goal is to turn raw personal fitness data into clearer historical analysis that helps the user make better training decisions.

FitLens will focus on three main categories of insight: recovery trends, fitness trends, and training progression. Recovery trends will look at historical signals such as HRV, resting heart rate, sleep, and recent training load. Fitness trends will track longer-term improvement using data such as VO2 max, workout consistency, and strength progress. Training progression will analyze volume, intensity, exercise performance, plateaus, and possible overreaching patterns. Instead of saying, "You are ready today," FitLens will say things like, "Last month your volume increased while HRV trended downward, so next month you may want to hold volume steady or add more recovery."

The application will be local to the user's machine. That means the user's health and workout files stay on their computer instead of being uploaded to a web server. For the

initial version, the app could minimally run as a command-line interface where the user provides file paths for the Apple Health XML export and Hevy CSV export, then receives a monthly report in the terminal and exported chart files. The ideal finished version would use a Python desktop GUI, likely PySide6 or Tkinter/CustomTkinter, where the user can select files, import data, view historical graphs, compare months, and generate a monthly coaching summary.

Major Python packages may include pandas for tabular data processing, lxml or xml.etree.ElementTree for parsing Apple Health XML, matplotlib or plotly for charts, sqlite3 for local storage, and a GUI library such as PySide6, Tkinter, or CustomTkinter. A later version could use APIs if available, but version 1 will stay focused on local file-based imports and historical analysis.

2. Task Vignettes

Vignette 1: Importing Apple Health and Hevy Data

A user opens FitLens after exporting their Apple Health data and Hevy workout history. In the first version, they run a command such as `python fitlens.py import --apple export.xml --hevy workouts.csv`. In the future desktop version, they would open the app, click an Import Data button, choose the Apple Health XML file, choose the Hevy CSV file, and then click Process. FitLens reads both files, checks whether they are valid, extracts the needed information, and stores a cleaned version of the data locally.

After the import finishes, the user gets a simple confirmation message that tells them how many workouts were imported, how many health records were found, and whether any data could not be matched. This gives the user confidence that their files were processed correctly before they start looking at monthly trends or recommendations.

Technical Details

The user will upload two main data files:

1. Apple Health XML export
2. Hevy CSV workout export (CSV or JSON depending on final data form)

Apple Health data extraction

The Apple Health parser will attempt to extract health and recovery-related data, including:

- Heart rate records
- Resting heart rate
- Heart rate variability, if available
- Sleep records, if available

- VO2 max, if available
- Body weight and other body stats, if available

Heavy workout data extraction

The Heavy parser will extract strength training data using their official schema:

```
Workout{  
  id                [...]   
  title             [...]   
  routine_id        [...]   
  description        [...]   
  start_time         [...]   
  end_time           [...]   
  updated_at         [...]   
  created_at         [...]   
  exercises          [...]   
}
```

Data cleaning and storage

After both files are parsed, the app will clean and organize the data into pandas DataFrames. The cleaned data will then be saved locally so the user does not need to reprocess the same files every time.

Local storage may use:

- SQLite for structured long-term storage
- CSV cache files for simpler version 1 storage

Error handling

The system should clearly notify the user when something goes wrong, such as:

- A required file is missing
- The file format is invalid
- A file is unsupported
- Expected fields are missing

- Certain optional data, such as sleep or VO2 max, is not available

This keeps the user informed without making them inspect the raw export files manually.

Vignette 2: Matching Workout Logs With Heart-Rate Data

After the user imports their data, FitLens automatically connects each Hevy workout with Apple Health heart-rate readings from the same time period. For example, if Hevy says the user lifted from 5:30 PM to 6:25 PM, FitLens searches Apple Health for heart-rate records inside that time window. The app then calculates the average heart rate, max heart rate, and estimated intensity for that workout.

This step matters because workout logs alone do not show how hard the workout may have been on the body. A workout with the same volume could be associated with different heart-rate patterns depending on fatigue, conditioning, or the type of training being done. By combining Hevy performance data with Apple Health heart-rate data, FitLens can create better historical summaries of training stress and recovery patterns.

Technical Details

Inputs

This part of the app uses two cleaned datasets:

- Cleaned Hevy workout DataFrame
- Cleaned Apple Health heart-rate DataFrame

Workout and heart-rate matching

The app will match each Hevy workout with Apple Health heart-rate records from the same time period.

The main matching method will use:

- Workout start timestamp
- Workout end timestamp
- Apple Health heart-rate timestamps

If exact start and end times are available, the app can directly match heart-rate records that occurred during the workout.

Workout-level outputs

For each workout, the app may calculate:

- Average heart rate

- Maximum heart rate
- Time spent in broad heart-rate intensity zones, if enough data is available
- Whether heart-rate data was successfully matched
- Whether the match was exact or estimated

Edge cases

The app should handle common data issues, including:

- Missing workout duration
- Missing workout start or end time
- Time zone mismatches between Apple Health and Hevy data
- Gaps in Apple Health heart-rate records
- Multiple workouts on the same day
- Workouts with no matching heart-rate data

When matching is uncertain, the app should label the result clearly rather than presenting it as fully reliable.

Vignette 3: Generating a Monthly Coaching Report

At the end of a month, the user exports updated Apple Health and Hevy files and imports them into FitLens. Then they run a command such as `python fitlens.py report --month 2026-06` or select a month from the future desktop interface. FitLens generates a monthly check-in report that summarizes training consistency, workout volume, strength progression, cardio/heart-rate patterns, sleep and recovery trends, and any notable changes compared to previous months.

Instead of giving the user a single daily readiness score, the report explains the month as a training block. For example, it might say that workout consistency improved, lower-body volume increased, HRV trended down, and resting heart rate was slightly higher than the user's usual baseline. The recommendation might be to keep frequency steady next month, avoid another large volume jump, and prioritize sleep or deloading if fatigue markers continue moving in the wrong direction.

Technical Details

Inputs

This part of the app uses the cleaned data that was already imported and organized from Apple Health and Hevy.

The main inputs include:

- Cleaned Apple Health health metrics
- Cleaned Hevy workout metrics
- Workout sessions matched with heart-rate data from the same time windows
- Historical data from previous months

Monthly report outputs

The app will generate a monthly fitness report that summarizes the user's training patterns and progress.

The monthly report may include:

- Training consistency score
- Progression score
- Recovery trend score
- Overall month summary
- Main positive trend
- Main warning signal
- Recommendation for the next month

Recovery trend analysis

Instead of giving a daily readiness score, the app will look at recovery trends across the month.

This may include:

- HRV trend over the month
- Resting heart rate trend over the month
- Sleep duration or sleep quality, if available
- Training load trend
- Relationship between training intensity and recovery metrics

Recommendation logic

The app should use the monthly trends to generate simple coaching-style recommendations.

For example, it may suggest:

- Increasing training consistency
- Reducing volume if recovery markers are trending poorly
- Maintaining the current plan if progress and recovery look balanced
- Adding more cardio if cardiovascular metrics are not improving
- Watching for a plateau if strength volume or performance has stalled

Missing data handling

Some Apple Health exports may not include every metric. For example, a user may not have sleep, HRV, or VO2 max data.

When data is missing, the app should still generate the report, but it should clearly explain which parts of the recommendation are limited because certain data was unavailable.

Vignette 4: Reviewing Historical Charts and Training Trends

After generating the monthly report, the user wants to understand what changed over time. They open the trends section, or in version 1 they generate chart image files from the command line. FitLens shows graphs for workout volume, strength progress on selected lifts, resting heart rate, HRV, VO2 max, and training consistency. The user can compare the last month against the previous month, the last 90 days, or their full imported history.

For example, the user might notice that their bench press volume increased quickly over three weeks while HRV dropped and resting heart rate increased. FitLens would not claim a medical conclusion, but it could flag that this pattern may suggest accumulated fatigue or recovery stress. This helps the user make a more informed decision about whether to keep pushing, maintain, or reduce intensity next month.

Technical Details

Inputs

This part of the app uses the user's historical fitness data, including:

- Historical Hevy workout data
- Historical Apple Health metrics
- Calculated monthly summaries from the app's analysis system

Visualization outputs

The app will turn the historical data into visual summaries that help the user understand long-term patterns.

The outputs may include:

- Line charts
- Bar charts
- Summary tables
- Workout consistency calendars
- Trend labels such as:
 - Improving
 - Stable
 - Plateauing
 - Possible overreaching

Possible chart types

The app may generate charts such as:

- Weekly training volume
- Monthly workout count
- Estimated one-rep max trends
- Resting heart rate trend
- HRV trend
- VO2 max trend
- Workout consistency calendar
- Monthly score history
- Exercise-specific progression charts

CLI version

In the command-line version, the user will run a Python command that generates the visual reports.

The CLI version can export charts as image files, such as:

- PNG charts
- CSV summary tables

- A monthly report folder containing the generated outputs

GUI version

In the desktop GUI version, the same charts and summaries can be displayed directly inside the app.

The GUI may allow the user to:

- Select a month to review
- View monthly summary scores
- Open individual charts
- Compare the current month against previous months
- Read trend labels and recommendations in plain language

Vignette 5: Comparing Months and Getting Recommendations Going Forward

After several months of imports, the user wants to know whether their training is actually improving. FitLens compares the current month to past months and shows what changed. It may show that total volume increased, workout consistency improved, and strength on major lifts moved up. It may also show that recovery markers got worse or that progress slowed despite more training volume.

The user receives plain-language recommendations for the next month. For example, FitLens might recommend increasing volume slightly if progress is stable and recovery trends look normal. It might recommend maintaining volume if progress is good but recovery is starting to decline. It might recommend a deload or reduced intensity if training load jumped sharply while sleep, HRV, or resting heart rate moved in a concerning direction. These recommendations are based on historical patterns, not real-time readiness.

Technical Details

Inputs

The recommendation system uses the monthly data already calculated and stored by the app.

The main inputs include:

- Monthly summaries from the local database
- Current month training and recovery metrics
- Previous month metrics

- Rolling average metrics from earlier months

Comparison process

The app compares the current month against both recent history and longer-term trends.

The comparison may include:

- Current month vs. previous month
- Current month vs. rolling average
- Training volume changes
- Workout consistency changes
- Strength progression changes
- Recovery trend changes

Recommendation outputs

The app will generate a simple coaching-style recommendation for the next month.

Each recommendation may include:

- Month-over-month comparison
- Recommendation category
- Plain-language explanation
- Main reason for the recommendation
- Suggested focus for the next month

Possible recommendation categories

The recommendation category may be one of the following:

- Increase: training appears to be going well, so the user may be able to progress slightly
- Maintain: training and recovery look balanced, so the user should keep the current approach
- Deload: training stress may be too high, so the user should reduce volume or intensity temporarily
- Investigate: the data shows an unusual pattern or missing information that needs closer review

Example recommendation logic

The first version of the recommendation engine should use simple, transparent rules.

Examples:

- If performance improves and recovery is stable, recommend continuing the current plan or making a small progression.
- If performance stalls while training volume rises, recommend checking recovery, exercise selection, or considering a deload.
- If recovery metrics worsen after a large volume increase, recommend holding or reducing volume.
- If workout consistency drops, recommend focusing on schedule adherence before increasing intensity.
- If important data is missing, explain that the recommendation is limited.

Version 1 approach

The recommendation engine should stay simple and understandable in version 1. Instead of using complex machine learning, the app should rely on clear rule-based logic so the user can understand why each recommendation was generated.

Vignette 6: Refreshing Data Over Time

A few weeks later, the user exports new Apple Health and Hevy files and imports them into FitLens again. The app recognizes that some workouts were already imported and only adds or updates the new records. The user's local history grows, and the app can generate better monthly comparisons because it now has more baseline data.

This workflow lets FitLens stay useful without needing live cloud integrations. The user remains in control of their files and can refresh their local database whenever they want. Over time, FitLens becomes more valuable because it can compare each month against the user's own history instead of using generic fitness assumptions.

Technical Details

Inputs

The refresh process uses newly exported files from the user's fitness apps.

The main inputs are:

- New Apple Health XML export
- New Hevy CSV workout export
- Existing historical data already stored by FitLens

Refresh process

When the user imports new files, the app should add the new data to the local database without creating duplicate records.

The refresh process may include:

- Parsing the new Apple Health export
- Parsing the new Hevy export
- Comparing new records against existing records
- Adding only new or updated records
- Recalculating monthly summaries
- Updating charts and trend labels
- Regenerating the latest monthly coaching report

Duplicate detection

The app should try to avoid storing duplicate workout records.

To identify possible duplicates, the app may compare:

- Workout dates
- Workout start and end times
- Workout names
- Exercise names
- Sets, reps, and weights
- Workout duration

If two records appear to describe the same workout, the app should keep one version rather than counting the workout twice.

Local data storage

The local database stores the user's historical imports over time. This allows FitLens to build month-to-month trends instead of only analyzing one upload at a time.

The database may store:

- Imported workout records
- Imported Apple Health metrics

- Matched workout and heart-rate records
- Monthly summaries
- Generated recommendation history

Future backup/export option

A future version of the app could include a backup or export feature for the local FitLens database. This would allow the user to save their processed data, move it to another computer, or restore it later if needed.

3. Crude Design Mockups / Wireframes

Version 1 CLI Flow

User runs command

```
python fitlens.py import --apple export.xml --heavy workouts.csv
```

Validate files -> Parse files -> Clean data -> Store locally

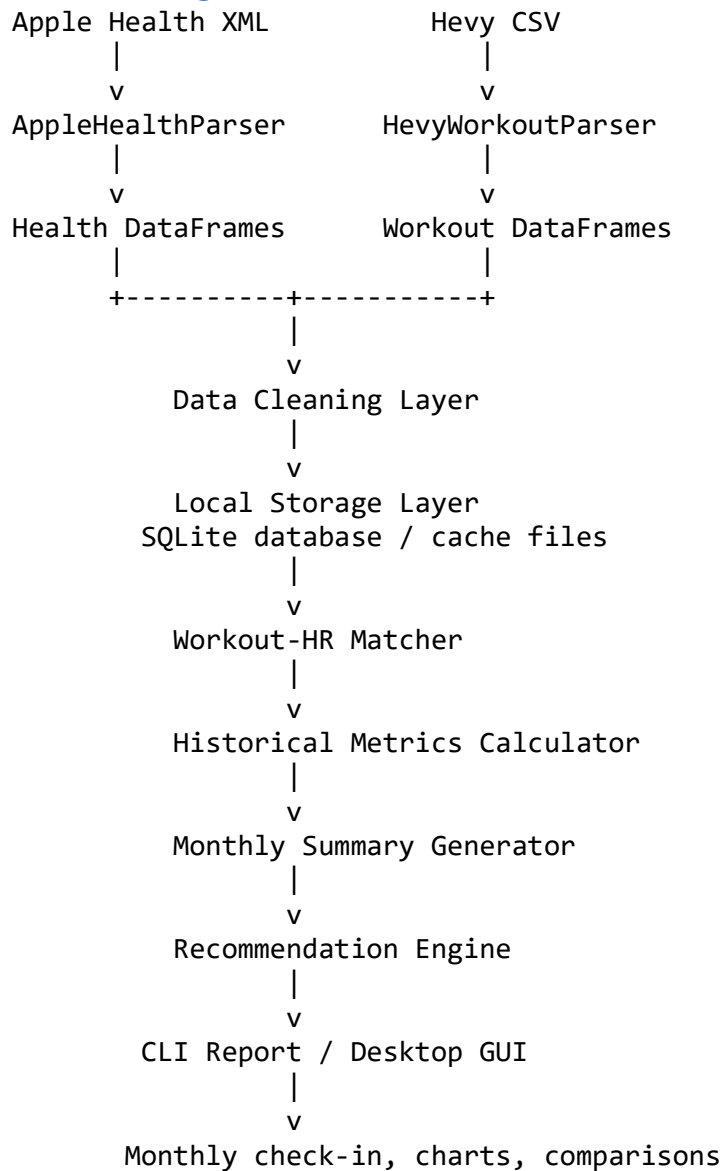
```
python fitlens.py report --month 2026-06
```

Monthly coaching summary + exported historical charts

4. Technical Flow

FitLens can be thought of as a local data pipeline. The user provides two raw files. The application parses those files, cleans them into a consistent structure, stores the useful data locally, calculates historical metrics, generates monthly summaries, and then displays or exports charts and recommendations.

Data Flow Diagram



5. Possible CLI Commands for Version 1

```
python fitlens.py import --apple export.xml --hevy hevy_workouts.csv
```

Imports and cleans both files, then stores the data locally.

```
python fitlens.py report --month 2026-06
```

Creates a monthly coaching summary for June 2026.

```
python fitlens.py compare --month 2026-06 --against previous
```

Compares the selected month against the previous month.

```
python fitlens.py charts --last 90 --output charts/
```

Exports historical trend charts as PNG files.

```
python fitlens.py status
```

Shows how much data has been imported and when the database was last refreshed.

May end up making the CLI more of a user friendly, step by step process invoked by a master script rather than specific commands.

6. Early Function/Class Ideas

Each function will have children functions so my code reads like a book and doesn't violate the SRP (Single Responsibility Principle).

```
class AppleHealthParser:
    def parse_xml(self, file_path):
        pass

    def extract_records_by_type(self, record_type):
        pass
```

```
class HeavyWorkoutParser:
    def parse_csv(self, file_path):
        pass

    def clean_workout_data(self):
        pass
```

```
class LocalDatabase:
    def save_health_records(self, dataframe):
        pass

    def save_workout_records(self, dataframe):
        pass

    def load_month_data(self, month):
        pass

    def load_historical_data(self):
        pass
```

```
class WorkoutHeartRateMatcher:
    def match_workouts_to_heart_rate(self, workouts_df, heart_rate_df):
```

```

        pass

class HistoricalMetricsCalculator:
    def calculate_training_volume(self, sets_df):
        pass

    def calculate_strength_trends(self, sets_df):
        pass

    def calculate_recovery_trends(self, health_df):
        pass

    def calculate_monthly_consistency(self, workouts_df):
        pass

class MonthlySummaryGenerator:
    def create_monthly_summary(self, metrics, month):
        pass

    def compare_to_previous_months(self, monthly_summary):
        pass

class RecommendationEngine:
    def generate_recommendation(self, monthly_summary, historical_baseline):
        pass

class ReportGenerator:
    def generate_terminal_report(self, monthly_summary):
        pass

    def export_charts(self, metrics, output_folder):
        pass

```

7. Final Self-Assessment

The biggest change from the original sketch is that FitLens is no longer planned as a web application and is no longer centered around a current readiness score. I originally imagined it more like a live dashboard with scores for the user's current state. After thinking through the bulk upload workflow, that framing seemed weaker because a user may only refresh their data monthly due to the chore of exporting and uploading their data. A monthly historical coaching report makes more sense because it fits the actual data flow better.

I feel moderately confident that I can implement the spec as written right now if I keep the first version focused on the command line. The CLI version mainly requires file parsing, data cleaning, basic calculations, historical comparisons, and simple reports. The desktop GUI is more ambitious, so I would treat that as the next step after proving that the data pipeline works correctly. If I feel comfortable with my data processing, I will spend more time on the GUI part of the application possibly skipping CLI altogether.

The biggest potential problem I need to solve is correctly matching Hevy workouts to Apple Health heart-rate data and then turning that data into fair recommendations. If the timestamps, time zones, or workout durations do not line up, the app may produce misleading results. Also, if the recommendation logic is too vague, the app will not feel useful. The project depends on making the monthly report simple, transparent, and grounded in the data.

The parts I am least familiar with are parsing large Apple Health XML exports efficiently (especially since they are formatted terribly in my opinion), designing fair historical scoring formulas, and building a polished desktop GUI in Python. I may need help deciding how simple or complex the monthly summary and recommendation formulas should be for version 1. I may also need help choosing the best GUI library once the command-line version is working. I tend to get in over my head trying to implement everything all at once. I may need to set realistic obtainable goals for myself.

This does have a little bit of similarity to a web application I developed called XPets: HealthQuest. That was a Nutrition and Fitness tracker mixed with a Tamagotchi RPG game that used Hevy workouts API. That backend was built with NodeJS but some parts can still apply to be ported over to Python.

8. AI Disclosure

I use AI (LLMs such as Claude, ChatGPT, Gemini) to brainstorm back and forth with as my “Rubber Ducky”. AI is also used to help “polish” my writing and fix any grammar/semantic issues.